

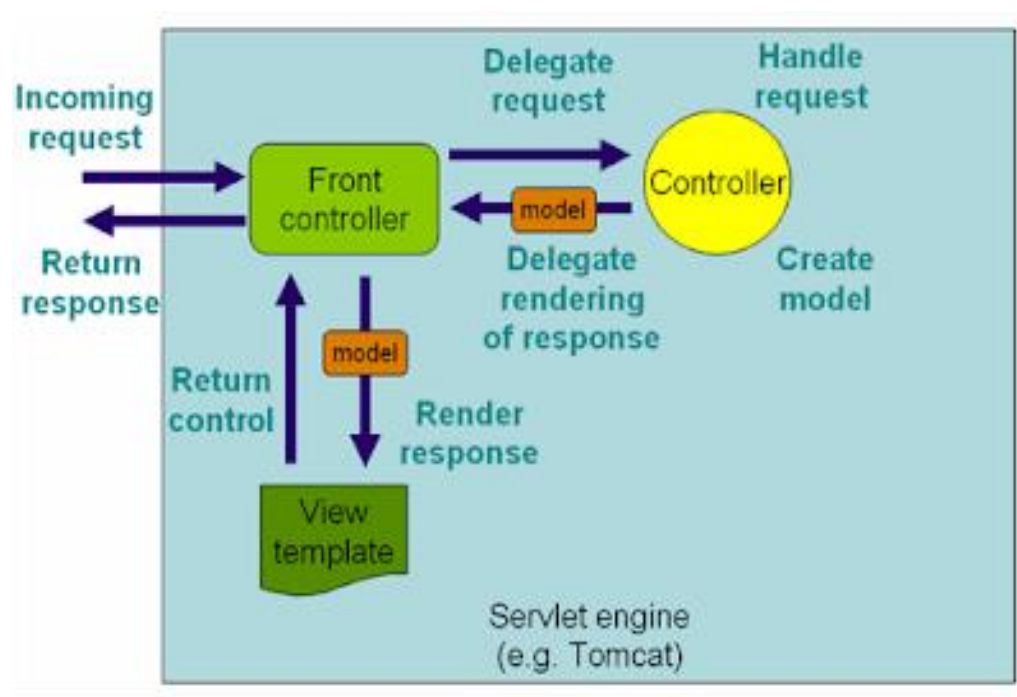
Spring MVC: -

- It is spring sub-framework of Spring framework which helps to build web application
- It follows MVC design pattern
- It is build on top of servlet API.
- Spring MVC implements all the features of spring core like IOC and DI.

MVC: -

- MVC stands for module view controller.
- This design pattern helps to build web applications.
- It is build on the top of servlet-API.
- MVC design pattern contains 3-important components
 1. Model.
 2. View.
 3. Controller.
- Spring MVC contains one more additional components Front-Controller.

Design Pattern:-Way to organize the code in our Application.



View Resolver:-

- View Resolver pattern is a J2EE pattern that allows a web application to dynamically choose its view technology, like HTML, JSP, Tapestry, JSF, XSLT, or any other view technology.
- In this pattern, the View resolver holds a mapping of different views, the controller returns the name of the view, which is then passed to View Resolver for selecting an appropriate view.

What is the difference between spring core and spring MVC:-

- The Spring MVC is part of the Spring framework, which helps you develop Java web applications using model web controller patterns. At the same time, Spring Core provides the Dependency injection and Inversion of Control. The Spring Container is part of Spring core.
- Both functionalities come in different JAR files. If you are developing just a core Java application using Spring, you just need Spring Core, but if you are creating a Web application, then you need spring-mvc.jar as well.

@RestController:-

- For RESTful web services, you can also create REST controllers by using the @RestController annotation, and in that case, you don't need to use the @ResponseBody and @controller annotation explicitly to tell Spring how it needs to respond.

Model:-

- Model consist application data it helps to transfer data from controller to view and view to controller.
- Model can be single object or multiple object.

View:-

- It is responsible to fetch the data from model and to display in specified format.
- Generally we use JSP+ JSTL but also spring provides supports for velocity , free marker , and thymeleaf.

Front-controller:-

- In spring MVC dispatcher servlet acts like front controller.
- All the incoming requests will be handled by front controller.

Controller:-

- Controller acts like intermediate layer between model and view.

- Controller is responsible for request processing.
- Controller maps model with view.

Why spring MVC:-

- Separate each roles model, view and controller..... etc. it will not mix the code .
- It is a sub framework of Spring framework so we can use the spring core features like IOC.
- Application will become loosely coupled and flexible..... etc.
- springMVC is easy to test.

@Controller:-

- It is a class level annotation .
- It helps to create instance of controller class.
- @controller is child of @component.

@RequestMapping:-

- It can be used as both class level and method level.
- We use this annotation on class level to define base url.
- We can use this annotation on top of method to define method specific url.

@RequestParam:-

- Request param will help to fetch data from form fields or from url query Strings.

@ModelAttribute:-

- Model attribute will help to convert form data to java object.

Model :-

- Model is interface present in spring framework container is responsible to create Model object
- Model acts like a container which helps to transfer data from controller to view.
- Model interface contains add Attribute method which accepts two arguments as First argument we have to pass key in the form of string And Second arguments we have to pass value in the form of object.

Model And view:-

- It is a class present in spring framework
- It is capable of Transfer the data by targeting a particular View.
- We can set View either by using constructor or method.

- Set ViewName is a method which accepts String data as argument we have to pass view name as argument.
- To set the data he have to invoke add Object() by passing key as argument one and value as argument Two.

internal Resourcer ViewResolver:-

- It is class present in spring framework
- This class generally used from specified view technology and file location
- Set Suffix method accepts one String parameter as arguments we have to pass View extension as argument
- Set prefix method also accepts one String parameter as arguments we have to pass view location as argument.
- If view pages are available inside web app then need not to specify view location by using set prefix.

Step create a Spring MVC project

1. Create a Maven Web Project

- File → New → Maven Project
- Choose **Webapp** as archetype → maven-archetype-webapp
- Group ID: com.example
- Artifact ID: springmvc-demo
- Finish

2. Add Spring MVC Dependencies

```
<dependency>

    <groupId>org.springframework</groupId>

    <artifactId>spring-webmvc</artifactId>

    <version>5.3.34</version>

</dependency>
```

3. Directory Structure

Make sure the folder structure looks like

src/main/java

src/main/webapp/WEB-INF

src/main/webapp/WEB-INF/views

4).Create WebInitializer (Java-based config instead of web.xml)

The `WebApplicationInitializer` is used to replace the traditional `web.xml` file in Spring-based web applications.

Purpose / Use:

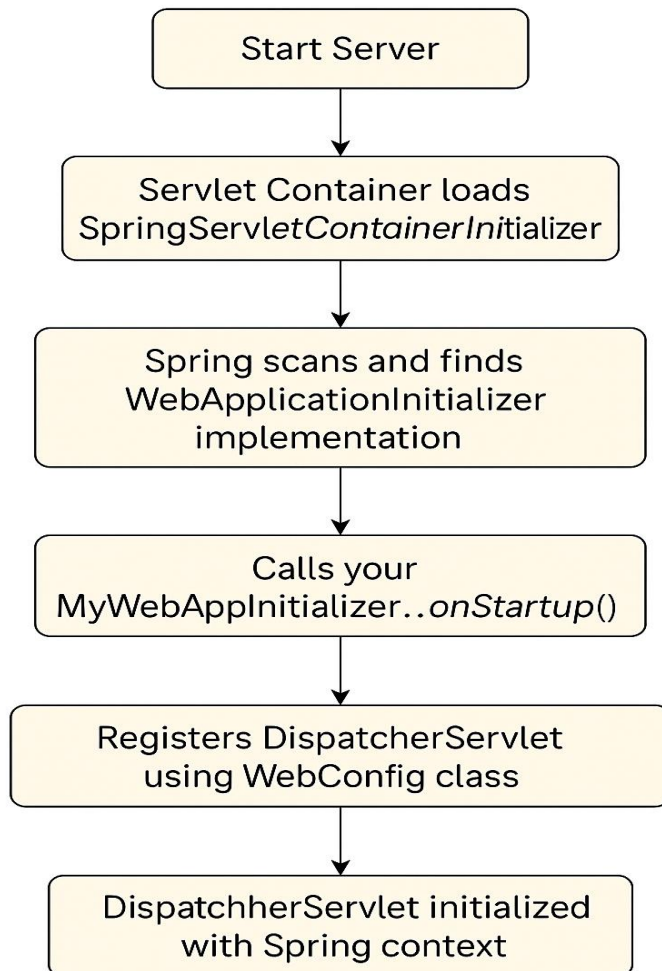
1. **Bootstraps the Spring MVC `DispatcherServlet`** programmatically (without XML).
2. **Registers configuration classes** (`@Configuration`) for the application.
3. **Defines servlet mappings**, context parameters, filters, etc.
4. **Starts the application in Java-based (class-level) configuration style** —
5. recommended from Spring 3.1+.

When your application starts:

- The servlet container (like Tomcat) looks for classes implementing `WebApplicationInitializer`.
- Spring detects your initializer and runs it.
- It sets up the **`DispatcherServlet`** with the provided `@Configuration` class.
- This bootstraps the Spring MVC framework.

Working Flow of `WebApplicationInitializer`

Working Flow of WebApplicationInitializer



```

package com.jspider.config;

import org.springframework.web.servlet.support.AbstractAnnotationConfigDispatcherServletInitializer;

public class WebInitializer extends AbstractAnnotationConfigDispatcherServletInitializer {

    {
        System.out.println("WebInitializer");
    }

    @Override
    protected Class<?>[] getRootConfigClasses() {
        // TODO Auto-generated method stub
        return null;
    }

    @Override
    protected Class<?>[] getServletConfigClasses() {
        // TODO Auto-generated method stub
        return new Class[] {StudentConfig.class};
    }

    @Override
    protected String[] getServletMappings() {
        // TODO Auto-generated method stub
        return new String[] {"/"};
    }

}

```

5. Create Spring Configuration Class

```

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.view.InternalResourceViewResolver;

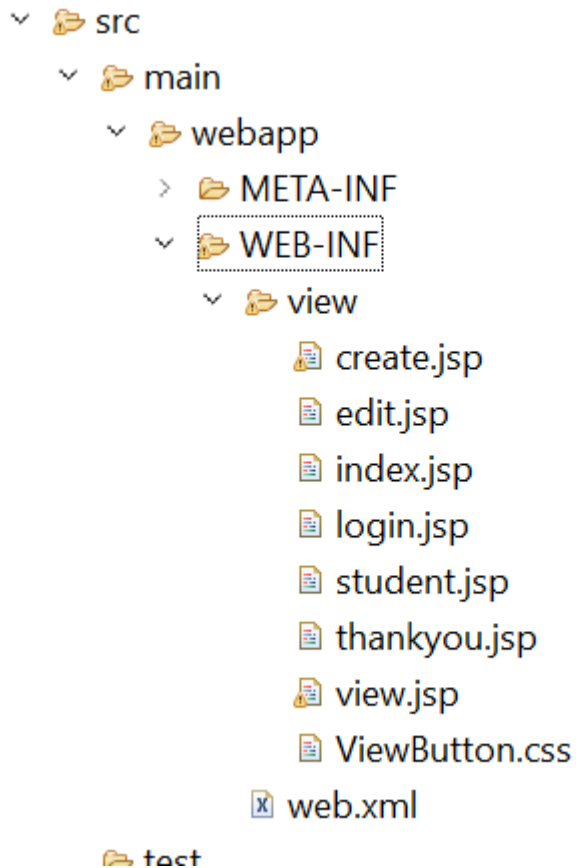
@Configuration
@ComponentScan(basePackages = "com.jspider")
public class StudentConfig {

    @Bean
    public InternalResourceViewResolver viewResolver() {
        InternalResourceViewResolver views = new InternalResourceViewResolver();
        views.setPrefix("/WEB-INF/view/");
        views.setSuffix(".jsp");
        return views;
    }

}

```

View file Structure



6. Create Controller Class

```
@Controller
public class StudentController {
    @Autowired
    StudentDao dao;

    @PostMapping("/save")
    public ModelAndView saveStudent(ModelAndView m, Student s) {
        System.out.println("SaveStudent() from Controller");
        dao.saveStudent(s);
        m.addObject("std", dao.getAllStudent());
        m.setViewName("view");
        return m;
    }
}
```

7. Create JSP Page

File: src/main/webapp/WEB-INF/views/view.jsp

```
<%@ page contentType="text/html; charset=UTF-8" %>
<html>
<head>
    <title>Spring MVC Home</title>
```



```
</head>
<body>
  <h1>Welcome to Spring MVC with Java Config!</h1>
</body>
</html>
```

8. Run the Project

1. Right-click project → Run As → Run on Server
2. Use Tomcat Server
3. Visit: <http://localhost:8080/springmvc-demo>

1. Model and View (ModelAndView)

◆ What it is:

A class that holds both:

- **Model** (data you want to send to the view)
- **View** (name of the view page)

```
@GetMapping("/welcome")
```

```
public ModelAndView showWelcome() {
```

```
    ModelAndView mv = new ModelAndView("welcome"); // view name: welcome.jsp
```

```
    mv.addObject("message", "Welcome to Spring MVC!");
```

```
    return mv;
```

```
}
```

2. Model

What it is:

An **interface** provided by Spring to **add attributes** (key-value pairs) to the model, which are accessible in the view (like JSP or Thymeleaf).

```
@GetMapping("/home")
```

```
public String showHome(Model model) {
```

```
    model.addAttribute("username", "Hari");
```

```
    return "home"; // home.jsp
```

```
}
```

3. ModelMap

◆ What it is:

A **Map-like** object provided by Spring (extends `LinkedHashMap`) that stores attributes and works like `Model`

```
@GetMapping("/about")

public String aboutPage(ModelMap modelMap) {

    modelMap.addAttribute("name", "hari");

    return "view"; //view.jsp

}
```

What is Expression Language (EL)?

Expression Language (EL) is a simple and powerful **expression syntax** used in **JSP** and **Spring (especially with Thymeleaf or JSP)** to access **data stored in objects** like JavaBeans, maps, lists, or request/session attributes.

Purpose of EL:

- To **access model data easily** from the view layer (like JSP/Thymeleaf)
- To avoid using complex Java code in JSP
- To improve readability and simplicity of templates

Basic Syntax:

```
${expression}
```

Use Cases in JSP:

If your controller sets:

```
model.addAttribute("username", "Hari");
```

Accessing Model Attributes

<h1>Hello, \${username}</h1>

Working with Collections

<c:forEach items="\${students}" var="s">

\${s.name}

</c:forEach>

Feature	Example	Description
Property access	<code>\${student.name}</code>	Access bean property
List access	<code>\${students[0]}</code>	Access list element
Map access	<code>\${map['key']}</code>	Access map value
Arithmetic	<code>\${price * quantity}</code>	Do basic arithmetic
Logical operators	<code>\${age > 18}</code>	Evaluate condition
Ternary operator	<code>\${score > 50 ? 'Pass' : 'Fail'}</code>	Simple if-else

Interview Questions

- What is Spring MVC?
- Explain the flow of a Spring MVC request.
- What is DispatcherServlet and its role?
- What are the main annotations used in Spring MVC?
- Difference between @Controller and @RestController?
- Difference between @RequestParam and @PathVariable?
- What is Model, ModelMap, and ModelAndView?
- How is view resolution done in Spring MVC?

1. What is Spring MVC?

Spring MVC (Model-View-Controller) is a web framework that is part of the Spring Framework. It is used to build web applications by separating the application logic into three components:

- **Model** – Contains the data and business logic.
- **View** – Represents the user interface (UI), often implemented using JSP, Thymeleaf, etc.
- **Controller** – Handles incoming requests, processes them using services or the model layer, and returns the view to be rendered.

Features of Spring MVC:

- Follows the **Front Controller** design pattern using `DispatcherServlet`.
- Supports annotation-based configuration.
- Provides clear separation of concerns.
- Integrates easily with REST APIs and form submissions.

2. Explain the flow of a Spring MVC request.

Here is the step-by-step flow of a Spring MVC request:

1. **Client sends a request** to a URL (e.g., `/home`).
2. The request is intercepted by `DispatcherServlet`, which acts as the **Front Controller**.
3. **HandlerMapping** determines which **Controller** should handle the request.
4. The selected **Controller** method executes the business logic and adds data to the **Model**.
5. The controller returns a **logical view name**.
6. **ViewResolver** resolves the logical view name to an actual view (e.g., JSP or Thymeleaf file).
7. The **view is rendered** with the model data and the response is sent back to the browser

3. What is `DispatcherServlet` and its role?

`DispatcherServlet` is the central component of Spring MVC and acts as the **Front Controller**. All HTTP requests pass through this servlet.

Main roles of `DispatcherServlet`:

- Intercepts all incoming requests.
- Uses **HandlerMapping** to identify the right controller.
- Passes the request to the appropriate **Controller**.
- Receives a view name and model from the controller.
- Uses **ViewResolver** to render the view.

- Returns the final response (usually HTML) to the client.

4. What are the main annotations used in Spring MVC?

Annotation	Description
@Controller	Marks a class as a controller component.
@RestController	Combines @Controller and @ResponseBody for REST APIs.
@RequestMapping	Maps web requests to handler methods.
@GetMapping / @PostMapping	Shortcut annotations for @RequestMapping with method type.
@RequestParam	Binds request parameters (e.g., query parameters) to method arguments.
@PathVariable	Binds URI template variables to method parameters.
@ModelAttribute	Binds form data or method return values to the model.
@ResponseBody	Binds method return value directly to the HTTP response body.
@Valid / @Validated	Used for validating form data or method arguments.

Difference between Spring MVC and SpringBoot

Spring MVC	Spring Boot
A web framework for building Java web applications using the Spring Framework.	A framework that simplifies Spring development using auto-configuration.
Requires manual configuration like <code>web.xml</code> , <code>DispatcherServlet</code> , and beans.	Provides auto-configuration and starter dependencies to reduce boilerplate.
Needs external servers like Tomcat or Jetty to deploy WAR files.	Comes with embedded servers (Tomcat, Jetty) and runs as a standalone JAR.
More setup time and complex project structure.	Quick to start with standard structure using Spring Initializr.
No built-in support for starter dependencies.	Offers various <code>spring-boot-starter-*</code> modules for rapid development.
Application must be deployed manually.	Can be run directly using <code>java -jar</code> command.
Suitable for developers who need full control over configuration.	Suitable for quick development, microservices, and RESTful web services.
Learning curve is higher due to detailed setup.	Easier for beginners and reduces development time significantly.

